

2주차

Software Development Cycle



2. Requirement Analysis

What is the goal of the program?	이 프로그램의 목표가 무엇인가?
Description of 'Input' and 'Output'	입력과 출력이 무엇인지 설명

3. Program Design

Algorithm Specification	알고리즘 설계(구체화)
Data Objects & Operation(ADT)	데이터 객체와 연산 방식(ADT)를 설계

4. Implementation

Coding & Debugging

5-6. Verification & Maintenance

Verifying Corecctness of the Program	프로그램이 올바르게 동작하는지 검증
Testing the program with a variety of input data	다양한 입력값으로 테스트

Algorithm

A **finite** set of instructions that accomplish a particular task.

- **Input** : the data to be used to produce output
- **Output** : the data produced by the algorithm
- **Definiteness** : Clear and unambiguous instructions 명령이 명확해야 함
- **Finiteness** : Terminate after a finite number of steps 유한해야 함.
- **Effectiveness** : Not unnecessary and redundant(중복적인) 효율적이어야 함.

Sorting

: Process of arranging data in a particular order, typically in **ascending or descending** order.

Selection Sort

Finds the **smallest element** and swaps it with the **first element**.

```
void selectionSort(int arr[], int n) {  
    int min_idx, temp;  
    for (int i = 0; i < n - 1; i++) {  
        min_idx = i;  
        for (int j = i + 1; j < n; j++) {  
            if (arr[j] < arr[min_idx])  
                min_idx = j;  
        }  
        temp = arr[min_idx];           //SWAP!  
        arr[min_idx] = arr[i];  
        arr[i] = temp;  
    }  
}
```

Bubble Sort

Repeatedly **swaps adjacent elements** if they are in the wrong order.

```
void bubbleSort(int arr[], int n) {
    int temp, swapped;

    for (int i = 0; i < n - 1; i++) {
        swapped = 0

        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];           //SWAP
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
        if (!swapped) break;           //한번도 교환을 안하면 swapped가 0 -> !0이므로 참 -> break
    }
}
```

Insertion Sort

Inserts each element into its correct position.

```
void insertionSort(int arr[], int n) {
    int key, j;

    for (int i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;               //key 놓기
    }
}
```

Merge Sort

Divides array, **sorts** subarrays, and **merges** them

```
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;           //n1은 왼쪽부분 개수
    int n2 = right - mid;               //n2는 오른쪽부분 개수
    int L[n1], R[n2];                  //각각 배열을 만들고
    for (int i = 0; i < n1; i++)        //arr의 왼쪽을 L[]에 복사해놓기
        L[i] = arr[left + i];
```

```

for (int i = 0; i < n2; i++)          //arr의 오른쪽을 R[]에 복사해놓기
    R[i] = arr[mid + 1 + i];
int i = 0, j = 0, k = left;
while (i < n1 && j < n2) {           // i가 왼쪽 개수보다 작고, j가 오른쪽 개수보다 작을 때까지
    if (L[i] <= R[j])                 // L[i]보다 R[j]가 작으면 L[i] 넣기 더 작은걸 넣어야 한다!
        arr[k++] = L[i++];
    else                             // 아니면
        arr[k++] = R[j++];
}
while (i < n1)                       //둘 중 하나가 먼저 끝나면 n1에 있는거 arr에 마저 넣기
    arr[k++] = L[i++];
while (j < n2)                       //n2에 있는거 arr에 마저 넣기
    arr[k++] = R[j++];
}
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;          // mid로 가운데 만들기
        mergeSort(arr, left, mid);                   //mid+1부터 right까지 재귀 호출
        mergeSort(arr, mid + 1, right);               //left부터 mid까지 재귀 호출
        merge(arr, left, mid, right);                 //merge 함수 호출
    }
}

```

Recursive Algorithm

An algorithm calls itself for subproblems. subproblem를 해결하기 위해 자기 자신을 호출하는 알고리즘

피보나치 코드

```

int fibonacci_recursive(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2);
}

```

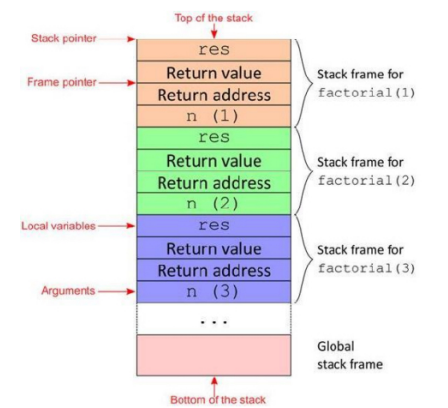
Advantages 장점 : Natural to represent algorithms 알고리즘 표현하기가 자연스럽다
 Clarity to reduce the time needed to write and debug
 코드가 명확해서 작성과 디버깅에 드는 시간을 줄일 수 있다

Disadvantages 단점 : More memory Usage 메모리 많이 사용함
 Time-consuming than its iterative counterpart
 반복문 방식보다 시간이 더 걸릴 수 있다

ADT Abstract Data Type

We **do NOT** want to perform different operations based on how the underlying data structure is constructed. 내부 구조를 알 필요가 없음.

Stack : LIFO Basic operation : push(), pop()
Queue : FIFO Basic operation : enqueue(), dequeue()



Performance Evaluation 성능 측정

어떻게 프로그램을 평가할까?

- Does the program meet the specifications of the task?
- Is the program's code readable?
- Does the program run efficiently? **How fast** does it run?
- **How much memory** space does the program require?

1. Machine-dependent Measurement

- The running time of the program. 프로그램의 실행 환경
- May differ by every single environment that the program runs in. 환경에 따라 다를 수 있음

2. Machine-independent Analysis

- **Space Complexity** of the Program
- **Time Complexity** of the Program

Space Complexity 공간복잡도

The **amount of space** taken by an algorithm to run as a function of its **input length**.

Fixed Space

Constant space that is **irrelevant** with the number of size of the program's inputs and outputs

프로그램의 입출력에 상관없이 항상 일정한 공간

e.g. Memory for instructions (code), constants, static variables, etc

메모리에서 Data, Code 영역은 고정 영역이다. 실행 중 사용량이 변할 수 있음.

Variable Space

Space that **depends** on the program's input size.

e.g. Memory for recursion stack, referenced variables 재귀호출 스택, 입력 크기에 따라 필요한 참조 변수

e.g. for문 sum()과 재귀함수 recursive_sum()의 공간복잡도 차이

```
int sum(int list[], int n){
    int i;
    int temp_sum = 0;
    for (i = 0; i < n; i++)
        temp_sum += list[i];
    return temp_sum;
}

int recursive_sum(int list[], int n){
    if (n)
        return (recursive_sum(list, n-1) + list[n-1]);
    return 0;
}
```

sum()의 공간복잡도 : list(array pointer), n(int), i(int), temp_sum(int) -> $4 \times 4 = 16\text{bytes}$

recursive_sum()의 공간복잡도 : Fixed - list(array pointer), n(int) -> $2 \times 4 = 8\text{bytes}$

Variable - list(array pointer), n(int), return_address -> $3 \times 4 = 12\text{bytes}$

Time Complexity 시간복잡도

The amount of time taken by an algorithm to complete its process as a function of its input length

입력 크기(input length)에 따라, 알고리즘이 작업을 끝내는 데 걸리는 시간의 양

The number of primitive operations or steps executed in the program 프로그램 안에서 연산의 횟수

sum()의 시간복잡도 : 변수선언 2회, loop condition check $n+1$ 번, +=연산 n 번, 리턴 1번 -> $2n+4$ 번 $O(n)$

recursive_sum()의 시간복잡도 : if문 $n+1$ 번, 재귀 return n 번, 마지막 return -> $2n+2$ 번 $O(n)$

Asymtotic Analysis 점근적 분석

: Evaluation the performance of an algorithm in terms of input size
입력 크기를 기준으로 알고리즘 성능 평가

Provides a high-level understanding of how an algorithm performs with regard to input size
Useful for comparing the efficiency of different algorithms for a specific problem

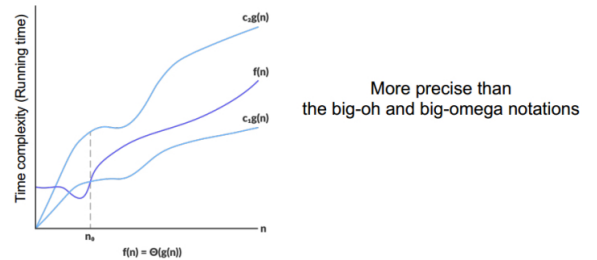
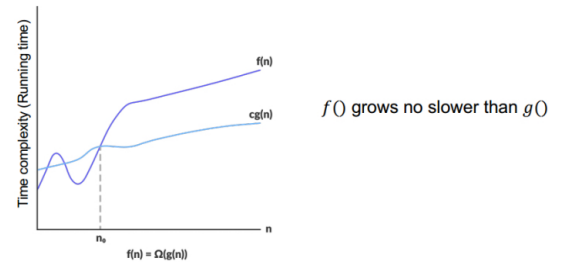
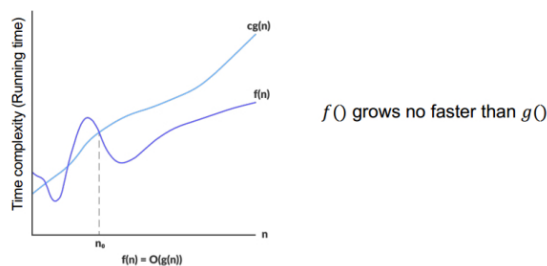
Predicting how an algorithm will perform on larger input sizes

Easy to perform with basic mathematical skills

Big-O notation

Upper bound of the running time (or space) of an algorithm

$f(n) = O(g(n))$ iff there exist positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n, n > n_0$



Big-Ω notation

Lower bound of the running time (or space) of an algorithm

Big-Θ notation

Upper & Lower bound of the running time (or space) of an algorithm

3주차 Array & Stack

Array

Fixed-size homogeneous collection of elements that are stored in a **contiguous(adjacent)** memory location.
연속된(인접한) 메모리 공간에 저장되는, 크기가 고정된 동일한 자료형 원소들의 집합.

A collection of elements, each identified by an **index or key**
각 원소가 인덱스(index) 또는 키(key)로 식별되는 원소들의 집합

All elements in an array are of the **same data type**. 모든 원소가 같은 데이터 타입
Homogeneous elements makes memory allocation **more predictable and efficient** since each element takes up the same amount of space. 메모리 할당을 더 예측 가능하고 효율적으로 만듦.

```
int arr[5] = {2, 4, 8, 12, 16}; //초기화
printf("Element at arr[2]: %d\n", arr[2]); //특정 인덱스 접근
printf("Element at arr[4]: %d\n", arr[4]);
arr[2] = 100; //특정 인덱스 값 수정
printf("Elements in Array: ");
for (int i = 0; i < 5; i++) {
    printf("%d ", arr[i]); //for문으로 traversing array
}
```

Multidimension

배열 변수는 “첫 번째 원소의 주소”를 담고 있다!

Multidimensional array는 차원이 늘어날수록 **traversal** 복잡도가 $O(n^2)$, $O(n^3)$ 처럼 커짐

배열을 함수에 넘길 때는 배열 이름이 첫 원소의 주소라는 사실과, 원본을 바꾸려면 **call by reference**로 주소를 넘겨야 함

함수에서 array 전달

```
void update_by_value(int e, int x){           //passed by value로 전달이 된다.
    e = x;
}
void update_by_reference(int list[], int i, int x){ //이렇게 해야 주소가 전달되어 바뀐!
    list[i] = x;
}
int main(){
    int arr[5] = {2, 4, 8, 12, 16};
    update_by_value(arr[3], 10);
    printf("Array elements: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    } }
```

C Array의 특징

인덱스가 0부터 시작됨. 첫 번째 인덱스가 0, 마지막 인덱스가 n-1번째

Contiguous Memory Location 연속된 메모리 공간에 저장됨

Address of the i-th element = Address of the 1st element + (index * size of each element)

Random Access

We can randomly access any element in the array using its index **$O(1)$**

랜덤 액세스라서 탐색 시간복잡도 $O(1)$ 이다!

C 배열의 한계

Array is Static

cannot be increased or decreased.

Array is Homogeneous

all the elements are the same data type. **Impossible** to manage **complicated data**

Insertion & Deletion are NOT EASY 삽입, 삭제가 쉽지 않음

insertion/Deletion을 하려면 2번의 과정을 거쳐야 함.

1. Traverse the array to find the element
2. Shift the remaining elements as per the operation $O(n)$

Dynamic Array

Dynamic Memory Allocation	StaticMemory Allocation
Memory is reserved at run time and we can allocate and deallocate	Memory is reserved at compile time and we cannot allocate and deallocate
It uses heap space	It uses stack space
It is more efficient and provides reusability of memory during run time	It is less efficient and NOT provide reusability of memory

malloc()	<pre>int * ptr = (int *) malloc (5 * sizeof(int));</pre> Allocate one continuous block of memory of argument size. ptr points to the starting address of this block NOT initialized. It may contain garbage values. Check <code>ptr != NULL</code> to ensure allocation	ptr이 이 블록의 시작 주소 가리킴 초기화 안됨! 쓰레기값 ptr != NULL로 할당 됐는지 확인
calloc()	<pre>int * ptr = (int *) calloc (5 , sizeof(int));</pre> Contiguous allocation of multiple blocks Allocates 5 blocks of memory, each of argument size. Unlike malloc(), calloc() initialize all block to 0 Use when you need zero-initialized memory	매개변수의 크기만큼 5개 만든다. 0으로 초기화! 0으로 초기화된 메모리 필요할때 사용
realloc()	<pre>int * ptr = (int *) realloc (5 * sizeof(int));</pre> Resize an already allocated memory block If realloc() fails, it returns NULL, and original blocked remains untouched. 만약 실패하면 NULL 리턴. 기존 배열 옆에 다른 값이 있으면 실패할 수 있음.	할당된 메모리 블록 크기 변경
free()	free(ptr) Deallocate memory that was previously allocated using malloc(), calloc(), or realloc().	

코드 week3 p34-p35 참고!

2-D array에서의 traverse 시간 복잡도 -> $O(\text{row} * \text{col}) == O(n^2)$

```
int row =3, col =4;
int** arr = (int**)malloc(row * sizeof(int*));    // row의 갯수만큼 포인터 배열을 만듦.
for (i = 0; i < row; i++)
    arr[i] = (int*)malloc(col * sizeof(int));      // arr마다 col의 개수만큼 배열을 만듦.
```

Struct 구조체

struct를 pass by value로 전달하는지 pass by reference로 전달하는지에 따라 다름.

```
void get_student_reference(struct student s){    //pass by value    값으로 넘김. 값 변경 안됨!
    s->student_id = 20240000;
    ...
}

void get_student_reference(struct student *s){    //pass by reference    주소로 넘김. 값 변경!
    s->student_id = 20240000;
    ...
}
```

```
get_student_value(s1);    //구조체 자체를 넘김
get_student_reference(&s1);    //구조체 주소를 넘김
```

```
s->student_id ==(*s).student_id
```

typedef

typedef를 하면 아래처럼 자료형처럼 쓸 수 있음.

코드 week3 p37-p38 참고

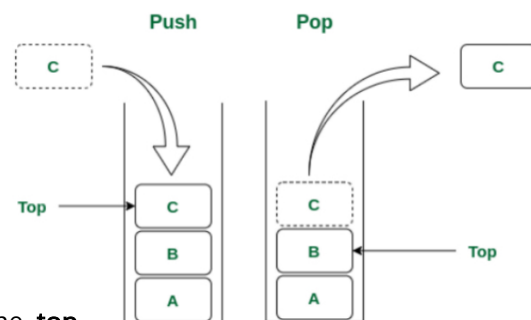
```
ptr_students = (student*)malloc(num_student * sizeof(student));
```

Stack

An ordered list where insertions and deletions are made at one end, called **top**
LIFO (Last-In First-Out) architecture

Push (insert) : Insert a new element only to the top of a stack.

Pop (Delete) : Delete an element only from the top of a stack.



Stack ADT

Object : A **finite** list with **zero or more elements**,

where insertions and deletions are made at **one end**, called the **top**

- **Stack Create** (*capacity*) ::= create an empty stack with the size of *capacity* **return** the stack
- **Stack Push** (*stack, item*) ::=
 - **if** (*isFull(stack)*) // the stack is full
 - **else** insert an item into the top of *stack* **return** *stack*
- **Element Pop** (*stack*) ::=
 - **if** (*isEmpty(stack)*) // the stack is empty
 - **else** remove the item at the top of the stack and **return** the item
- **Element Peek** (*stack*) ::=
 - **if** (*isEmpty(stack)*) // the stack is empty **else return** the item at the top of the stack
- **Boolean isFull** (*stack, capacity*) ::= **return** true **if** the stack is full, **else return** false
- **Boolean isEmpty** (*stack*) ::= **return** true **if** the stack is empty, **else return** false

```
typedef struct {
    int top;
    int capacity;
    int *array;
} Stack;          // stack이라는 자료형 정의!
```

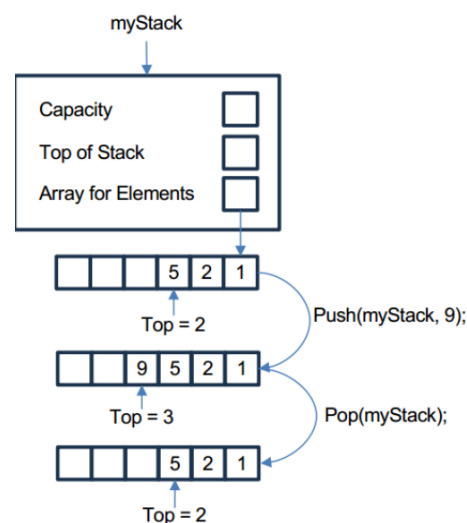
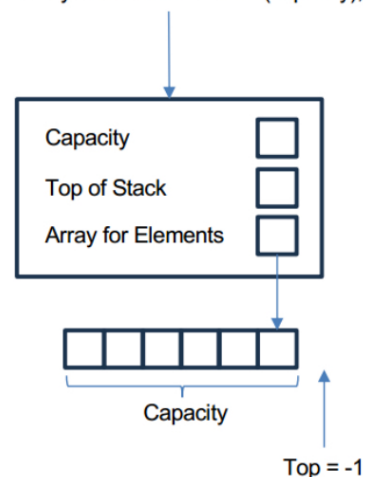
```
Stack * createStack(int capacity) {
    Stack *stack = (Stack *)malloc(sizeof(Stack));
    if (!stack)
        return NULL;    // out of space error
    stack->top = -1;
    stack->capacity = capacity;
    stack->array = (int *)malloc(stack->capacity * sizeof(int));
                                // for element data
    if (!stack->array)
        return NULL;    // out of space error
    return stack;
}
```

```
int isFull(Stack *stack) {          // Check if stack is full
    return stack->top == stack->capacity - 1;
}
```

```
int isEmpty(Stack *stack) {         // Check if stack is empty
    return stack->top == -1;
}
```

```
int peek(Stack *stack) {            // Return top element without removing it
    return stack->array[stack->top]; }
```

```
Stack *myStack = createStack(capacity);
```




```

int pop(Stack *stack) {           // Pop operation
    if (!isEmpty(stack))          // check whether the stack is empty
        return stack->array[stack->top--]; // 뺀 다음에 --를 해야 함
    printf("Stack is Empty!\n");
    return 0;
}

void push(Stack *stack, int new_element) { // Push operation
    if (!isFull(stack))                // check whether the stack is full
        stack->array[++stack->top] = new_element; //미리 ++을 하고 넣어야 함
    else
        printf("Stack is full!\n");
}

```

Palindromes 팰린드롬

A word/phrase/sequence that reads the same backward as forward

방법 1) Copy the characters of a word/phrase in a reverse order. Compare the characters in the same index
문자열을 뒤에서부터 복사해 원본이랑 비교

방법 2) Checking Palindromes using Stack 스택을 사용해서 판단

- 1) Push the first length/2 characters into the stack
- 2) Pop the element and compare it with the current character

```

int isPalindrome(Stack *stack, char word[], int length){
    int i;
    for (i = 0; i < length / 2; i++) { //반만 넣기!
        push(stack, word[i]);
    }

    if (length % 2 != 0) { // 홀수이면 하나 건너뛰기
        i++;
    }

    while (i < length) { //단어가 끝날 때까지
        if (pop(stack) != word[i]) { // 만약 단어가 다르면 return 0
            return 0;
        }
        i++;
    }
    return 1;
}

```

Bracket Check 괄호 체크

if it is an **opening bracket** (i.e., '(', '{', '[') then **push** it to stack. 열리는 괄호이면 push!
if it is a **closing bracket** (i.e., ')', '}', ']') then **pop** from stack 닫히는 괄호이면 pop!

Infix -> Postfix 변환

operand(피연산자)이면 결과에 두기

operator(연산자)이면

스택이 비어 있으면, 스택에 push

현재 연산자가 스택 top의 연산자보다 우선순위가 높으면 push

+,-위에 */ 쌓임

현재 연산자가 스택 top의 연산자보다 우선순위가 같거나 낮으면 pop 해서 prefix로

-> * / 위에 + - 는 그냥 못 올라가고, 먼저 * / 를 pop 하게 됨

+ - //이런 상황일 수 없다!

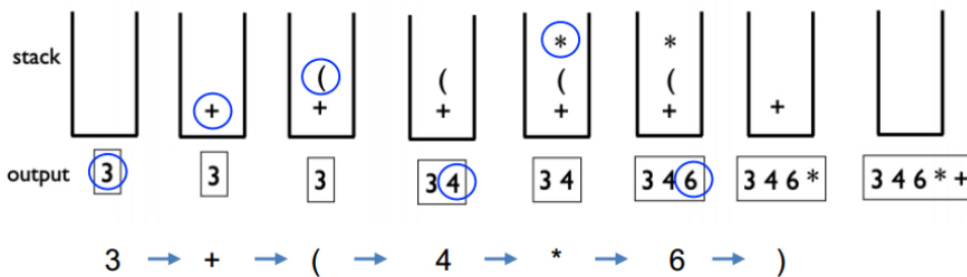
* /

(여는 괄호를 만나면, stack에 push

) 닫는 괄호를 만나면, (를 만날때까지 pop 하기

다 끝나면 순서대로 pop

$3 + (4 * 6) \longrightarrow 3 4 6 * +$



Upper & Lower bound of the running time (or space) of an algorithm

4주차 Queue

Queue

A list where **insertion** is done at **one end (tail)**, whereas **deletion** is made at the **other end (head)**.

FIFO (First-In First-Out) architecture

Enqueue (insert) : Insert a new element only to the tail of a queue.

Dequeue (Delete) : Delete an element only from the head of a queue.

Queue ADT

- Queue **Create** (*capacity*) ::= create an empty queue with the size of *capacity* **return** the queue
- Queue **Enqueue** (*queue*, *item*) ::=
 - **if** (*isFull*(*queue*)) // the queue is full
 - **else** insert an item into the tail of *queue* **return** *queue*
- Element **Dequeue** (*queue*) ::=
 - **if** (*isEmpty*(*queue*)) // the queue is empty
 - **else** remove the item at the head(front) of the queue and **return** the item
- Element **front** (*queue*) ::=
 - **if** (*isEmpty*(*queue*)) // the queue is empty **else return** the item at the front(head) of the queue
- Element **rear** (*queue*) ::=
 - **if** (*isEmpty*(*queue*)) // the queue is empty **else return** the item at the rear(tail) of the queue
- Boolean **isFull** (*queue*, *capacity*) ::= **return** true **if** the queue is full, **else return** false
- Boolean **isEmpty** (*queue*) ::= **return** true **if** the queue is empty, **else return** false

```
typedef struct {
    int front, rear;
    int capacity;
    int *array;
} Queue;
```

```
Queue *createQueue(int capacity){
```

```
    Queue *queue = (Queue*)malloc(sizeof(Queue));
```

```
    if (!queue)
```

```
        return NULL;
```

```
    queue->front = 0;
```

//front를 0으로 왜냐하면 이 코드에선 dequeue할 때 return하고 ++

```
    queue->rear = -1;
```

//rear는 -1로 !

```
    queue->capacity = capacity;
```

```
    queue->array = (int*)malloc(capacity * sizeof(int));
```

```
    if (!queue->array)
```

```
        return NULL;
```

```
    return queue;
```

```
}
```

```
int isFull(Queue *queue){
```

```
    return (queue->rear == queue->capacity - 1); }
```

```
int isEmpty(Queue *queue){
```

```
    return (queue->front == queue->rear); }
```

```
void enqueue(Queue *queue, int new_element){
```

```
    if (!isFull(queue)) // isEmpty가 거짓이면 -> 꽉 차있지 않으면
```

```
        queue->array[++queue->rear] = new_element; //먼저 더하고 enqueue. 자리를 먼저 만들어줘야 함.
```

```
    else
```

```
        printf("Queue is Full!\n");
```

```
}
```

```
int dequeue(Queue *queue){
```

```
    if (!isEmpty(queue)) // isEmpty가 거짓이면 -> 비어있지 않으면
```

```
        return queue->array[queue->front++]; // 먼저 return 시키고 front를 1더함.
```

```
    else
```

```
        printf("Queue is Empty!\n");
```

```
    return -1;
```

```
}
```

```
int front(Queue *queue){
```

```
    if (!isEmpty(queue)) // check whether the queue is full
```

```
        return queue->array[queue->front];
```

```
    printf("Queue is Empty!\n");
```

```
    return -1;
```

```
}
```

```
int rear(Queue *queue){
```

```
    if (!isEmpty(queue)) // check whether the queue is full
```

```
        return queue->array[queue->rear];
```

```
    printf("Queue is Empty!\n");
```

```
    return -1;
```

```
}
```

-> 배열 끝에 도달하면 앞쪽 빈칸을 재사용하지 못하는 문제가 있다!

Circular Queue

A list where insertion is done at one **end (tail)**, whereas deletion is made at the **other end (head)**, and the elements of the Queue are **arranged in a circle** rather than a straight line.

FIFO (First-In First-Out) architecture

```
Queue *createQueue(int capacity){
    Queue *queue = (Queue*)malloc(sizeof(Queue));
```

```
    if (!queue)
        return NULL;
```

```
    queue->front = 0, queue->size = 0;
```

//size라는 것을 새로 만들! empty와 full 구분 위함

```
    queue->rear = capacity - 1;
```

```
    queue->capacity = capacity;
```

```
    queue->array = (int*)malloc(capacity * sizeof(int));
```

```
    if (!queue->array)
        return NULL;
```

```
    return queue;
```

```
}
```

```
int isFull(Queue *queue){
```

```
    return (queue->size == queue->capacity); }
```

```
int isEmpty(Queue *queue){
```

```
    return (queue->size == 0); }
```

```
void enqueue(Queue *queue, int new_element){
```

```
    if (!isFull(queue)) {
```

```
        queue->rear = (queue->rear + 1) % queue->capacity;
```

```
        queue->array[queue->rear] = new_element;
```

// 늘리고 넣기

```
        queue->size += 1;
```

```
    }
```

```
    else
```

```
        printf("Queue is Full!\n");
```

```
    return;
```

```
}
```

```
int dequeue(Queue *queue){
```

```
    if (!isEmpty(queue)) {
```

```
        int element = queue->array[queue->front];
```

//빼고 늘리기!

```
        queue->front = (queue->front + 1) % queue->capacity;
```

```
        queue->size -= 1;
```

```
        return element;
```

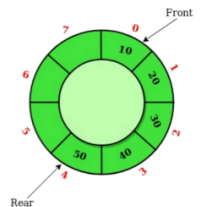
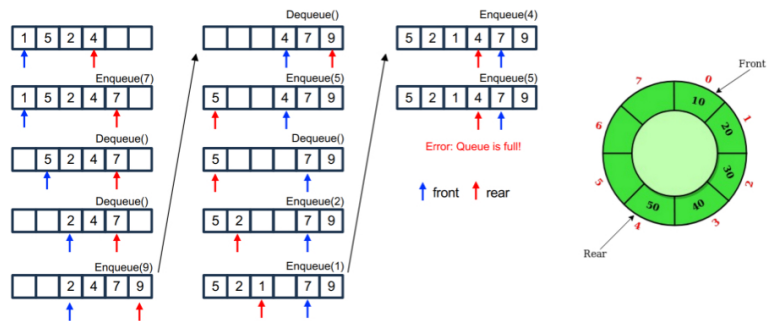
```
    }
```

```
    else
```

```
        printf("Queue is Empty!\n");
```

```
    return -1;
```

```
}
```

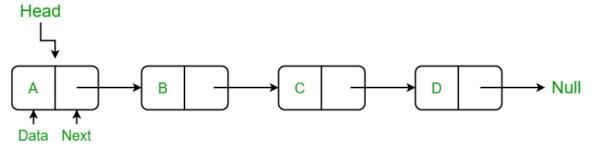


Linked List

A collection of nodes that **consecutively(연속적으로)** connected.

A node consists of an element and a pointer to the next node.

Nodes are **NOT necessarily adjacent in memory**



Dynamic Data Structure: The size of memory can be allocated/de-allocated at runtime.

Easy insertion and deletion : Insertion and deletion of elements in a linked list is easier than array

Efficient memory utilization: Allocate memory only when required.

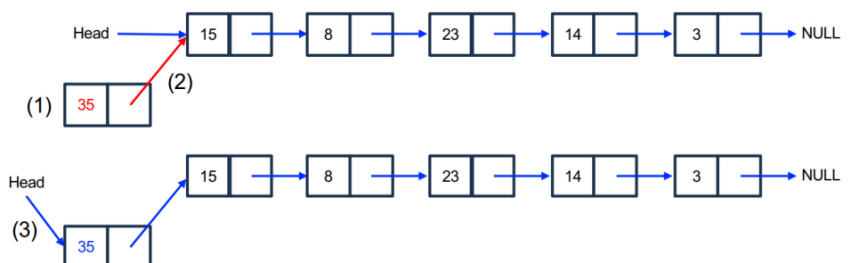
연산

- **Traversing** the linked list traverse하기
- **Searching** for an element in the linked list traverse의 한 종류. 하면서 비교
있으면 true 리턴, 없으면 false 리턴
- **Inserting / Deleting** an element from the linked list 포인터 연결 순서가 매우 중요하다! free()도 중요

```
void traverse(Node *head){
    Node *cur_node = head;
    while (cur_node != NULL){
        printf("%d ", cur_node->element);
        cur_node = cur_node->next;    //cur_node->next를 cur_node에 넣어준다.
    }
}
```

```
bool search_recursive(Node* head, int key){
    if (head == NULL)                // 끝났으면
        return false;
    if (head->element == key)         // key를 찾았으면
        return true;
    return search_recursive(head->next, key);    // 재귀적으로 다음 함수 탐색
}
```

```
bool search_iterative(Node* head, int key){
    Node *cur_node = head;
    while (cur_node != NULL){
        if (cur_node->element == key)
            return true;
        cur_node = cur_node->next;
    }
}
```



linked list INSERT

```
void insertAtHead(Node **head_ref, int new_data){    //포인터의 포인터! 포인터를 전달해야 하기 때문
    Node* new_node = (Node*)malloc(sizeof(Node));
    new_node->element = new_data;
    new_node->next = (*head_ref);
    (*head_ref) = new_node;
}
```

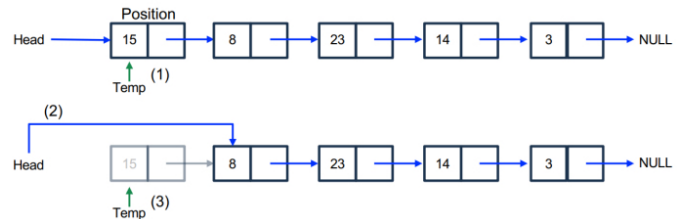
// 1. 새 노드 만들기
// 2. 새 노드의 next를 head_ref가 가리키는 곳으로!
// 3. head_ref에 new_node를 넣어줌!

```

void insertAtEnd(Node *head, int new_data){
    Node* new_node = (Node*)malloc(sizeof(Node));
    new_node->element = new_data;           // 새 노드 만들기
    new_node->next = NULL;                  //새 노드의 next를 NULL로 함
    Node *last = head;                     //last라는 노드를 만들고
    while (last->next != NULL)              //last->next가 NULL이 될때까지 탐색
        last = last->next;                 //last에 마지막 노드 주소 찾아서 저장
    last->next = new_node;                  //last->nex에 new_node 주소 넣기
}

void insertAfterPosition(Node *head, int new_data, int key){
    Node* new_node = (Node*)malloc(sizeof(Node));
    new_node->element = new_data;           // 새 노드 만들기
    Node *position = head;                 // position이라는 노드를 만들고,
    while (position->element != key)        //key가 나올 때까지 탐색
        position = position->next;         // new_node->next를 position->next으로 함
    new_node->next = position->next;        // position->next가 new_node를 가리키게 함.
}

```



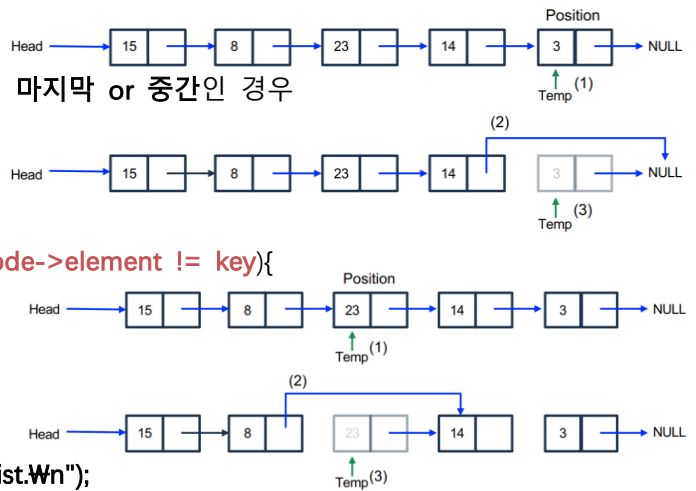
linked list DELETE

```

void deleteNode(Node** head_ref, int key){
    if (*head_ref != NULL){
        if ((*head_ref)->element == key){   //지우려는 것이 (*head_ref)->element 첫 번째 노드인 경우
            Node *temp = *head_ref;        //삭제할 데이터 보관용
            *head_ref = (*head_ref)->next;
            printf("Deletion %d of the first node.\n", temp->element);
            free(temp);
        }
        else {                               // 지우려는 것이 마지막 or 중간인 경우
            Node *prev_node = *head_ref;
            Node *cur_node = (*head_ref)->next;

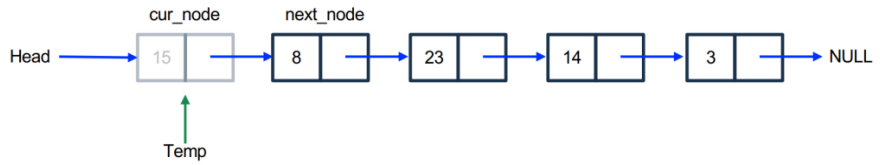
            while (cur_node->next != NULL && cur_node->element != key){
                prev_node = cur_node;
                cur_node = cur_node->next;
            }
            if (cur_node->element != key)
                printf("Deletion failed: No key in the list.\n");
            else{
                // cur_node->element가 key 인 경우. 찾은경우!
                Node *temp = cur_node;      //삭제할 데이터 보관용
                prev_node->next = cur_node->next; //cur_node->next를 pre_node->next에 넣는다!
                printf("Deletion %d of the middle/last node.\n", temp->element);
                free(temp);
            }
        }
    }
    else
        printf("List is Empty.\n");
}

```



전부 지우기

```
void deleteList(struct Node** head_ref){
    Node* cur_node = *head_ref;
    Node* next_node;
    while (cur_node != NULL) {
        next_node = cur_node->next;
        free(cur_node);
        cur_node = next_node;
    }
    *head_ref = NULL;
}
```



5주차 Linked List

	Array	Linked List
Pros & Cons	- Simple and easy to handle - Faster access to elements (random access) - Limited memory usage - Cannot deal with unpredictable input Expensive insert/delete	- Memory can be easily adjusted dynamically - Efficient insertion/deletion - Hard to handle Extra memory for references (pointers)
Traverse & Search	$O(n)$	$O(n)$
Insertion	$O(n)$ ($O(1)$ for the end)	$O(1)$
Deletion	$O(n)$ ($O(1)$ for the end)	$O(1)$
Random Access	O	X

Doubly Linked List

각 node가 next와 prev를 둘 다 가짐.

head의 prev는 NULL, 마지막 node의 next는 NULL.



장점

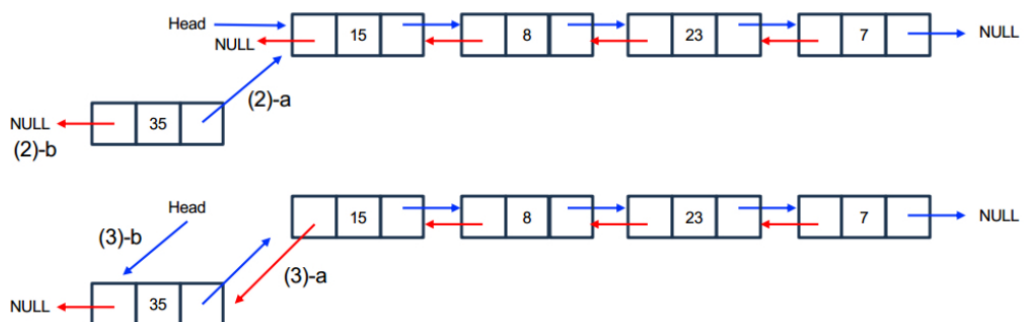
- Efficient **traverse** in both forward and backward direction
- Dynamic size** adjustment
- Flexibility**: Insert/Delete/Update the previous node if we do not have its address

단점

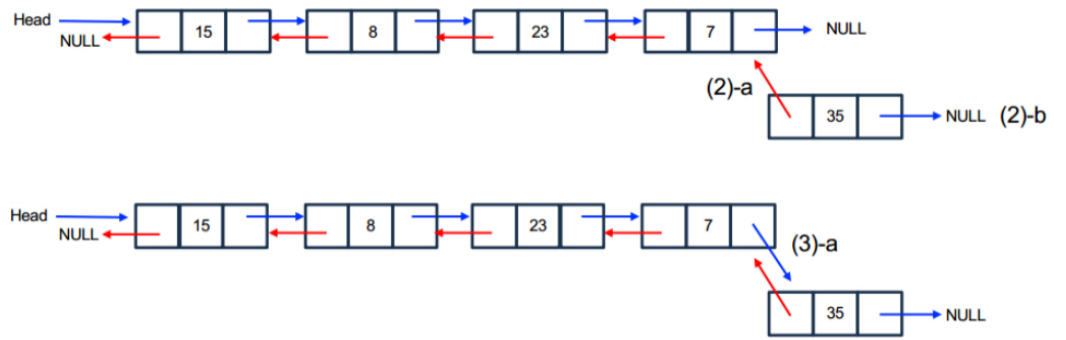
- More memory usage** for two pointers of each node
- Complex** pointer management

Insertion

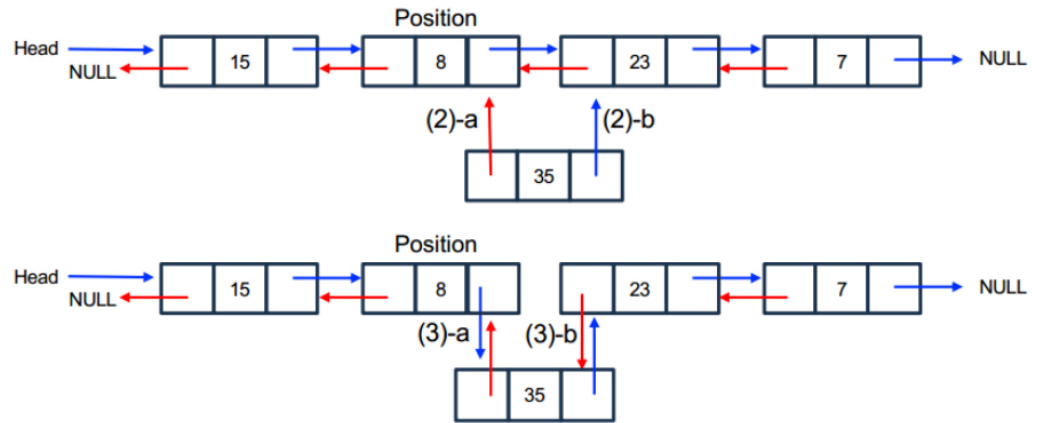
beginning



end

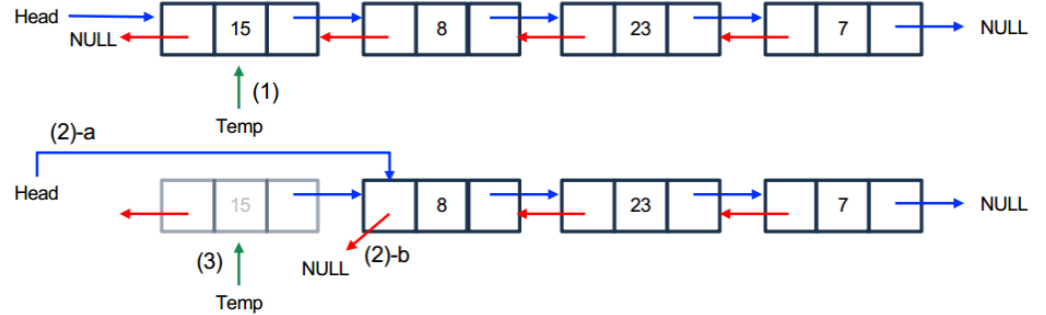


intermediate

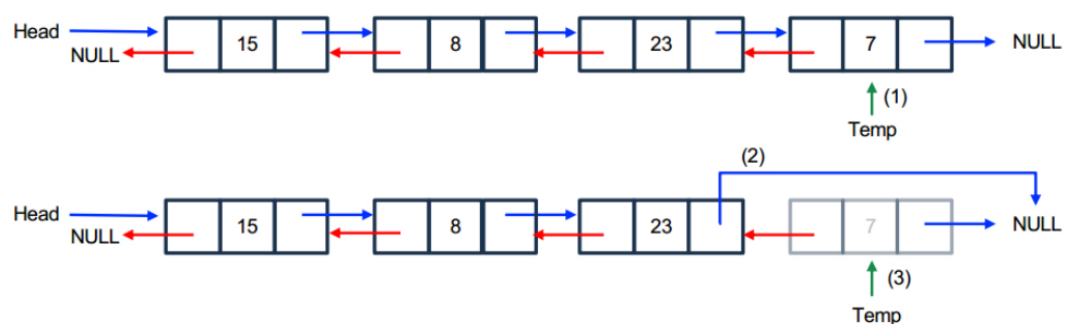


Deletion

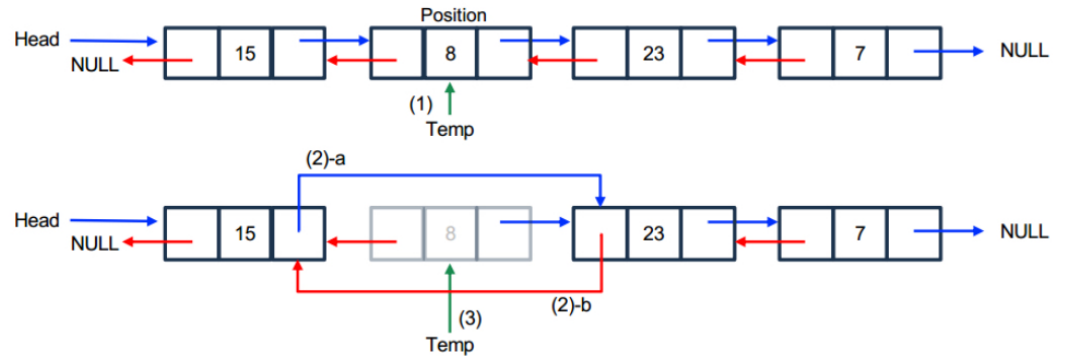
beginning



end



intermediate



Linked List 장단점

장점

Dynamic Data Structure: No need to estimate the total memory required.

Easy insertion and deletion : Insertion and deletion of elements in a linked list is easier than array

단점

More memory usage : Each node in a linked list stores data + a pointer (two for doubly linked list)

Pointer requires **additional memory**

Searching for an element requires linear traversal. $O(N)$

Poor **cache** performance (no spatial locality) 메모리가 실제로는 인접하지 않아서 발생함. 캐시를 못함

Even if the elements are sorted, you **cannot jump** to the middle

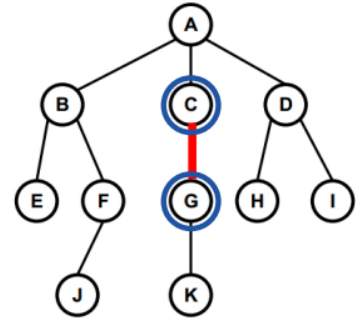
6주차 Tree

Tree

A **finite set** of one or more nodes connected by edges **without cycles**.

A **non-linear**, hierarchical data structure.

Has **one root node**, and each node can have child nodes.



용어 정리

Node : The node stores the data along with the links to other nodes.

Edge : The edge connects two nodes (parent-to-child) of a tree.

Parent & Child : If there is an edge from P to Q, P is a Parent of Q, and Q is a Child of P.

Siblings : Nodes that have the same parent (e.g., E and F are siblings)

Root : The first node of the tree without any parents (e.g., node A)

Leaf & Internal : If P has **no children**, P is a Leaf node. **Otherwise, it is an Internal node.**

자식이 없으면 leaf 노드, leaf 노드가 아니면 Internal 노드

빨간색 Internal 파란색 leaf

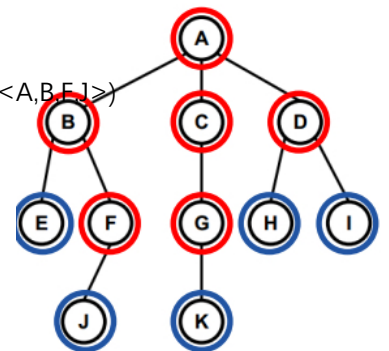
Path : A sequence of nodes from the start to destination. (e.g., Path A to J = $\langle A, B, F, J \rangle$)

시작부터 목적지까지의 경로

Ancestor & Descendant :

If there is a path from root to Q and P appears on the path,

P is an **Ancestor** of Q, Q is called a **Descendant**.



Degree :

Degree of a node is the **number of children** it has (e.g., degree of D is 2)

Degree of a **tree** is the maximum degree of the nodes (e.g., 3)

degree : 자식의 개수!

tree의 degree : 트리 안의 모든 노드 중 자식 수가 가장 많은 값

Level(depth) :

The level of a node is the length of the path from the root to that node

root부터 해당 노드까지의 길이

The level of tree is the maximum level of the nodes. (e.g., 4)

노드들 레벨 중 가장 큰 값

Width :

The width of a tree is the largest sum of the degrees at the same level (e.g., 5)

트리의 너비는 같은 레벨에 있는 노드들의 차수 합 중 가장 큰 값이다.

The size of a tree is the number of the nodes in the tree (e.g., 11)

Size : The **size of a tree** is the number of the nodes in the tree

트리의 크기(size)는 트리 안에 있는 노드의 개수

Subtree :

Tree T' rooted at a node P is a subtree of Tree T if T' has all descendants of P in T including P itself
트리 T에서 어떤 노드 P를 루트로 하는 T'가, P 자신과 P의 모든 자손을 포함하고 있으면 T'를 T의 부분트리(subtree) 라고 한다.

트리의 특성

Recursive Data Structure

Composed of smaller instances of the same data structure (i.e., **subarrays**)

같은 자료구조의 더 작은 형태들로 구성된다.

A tree has a root node, and every node has **zero or more** child nodes.

트리는 루트 노드를 가지며, 각 노드는 0개 이상의 자식 노드를 가진다.

Path from the root to node P

There exists exactly a single (unique) path from the root to each node

루트부터 노드 P까지는 unique한 경로가 있다!

Number of Edges in a Tree

The number of edges in a tree is always one less than the number of nodes.

트리의 edge 수는 항상 node 수보다 1개 적다.

$edge = node - 1$

Hierarchical Structure 계층 구조

Naturally representing many types of **hierarchical relationships**

다양한 종류의 계층 관계를 자연스럽게 표현할 수 있다.

Efficient Searching 효율적인 탐색

Searching for a specific data can be done in **$O(\log n)$ time**

특정 데이터를 찾는 작업을 $O(\log n)$ 시간에 할 수 있다.

Efficient Insertion & Deletion

Insertions/deletions can be done without additional complexity to search time

삽입/삭제는 탐색 시간에 비해 큰 추가 복잡도 없이 수행할 수 있다.

Dynamic & Flexibility

Tree can grow or shrink dynamically as needed

트리는 필요에 따라 동적으로 커지거나 줄어들 수 있다.

Binary Tree

A binary tree is a tree where every node has **0, 1, or 2 children**.

Each child node is labeled as either a **left child** or a **right child**.

Full Binary Tree

A binary tree where every node has 0 or 2 child nodes
자식 0개 또는 2개

Complete Binary Tree

A binary tree that has all levels completely filled with nodes except the last level.
마지막 level을 빼면 completely filled하다.

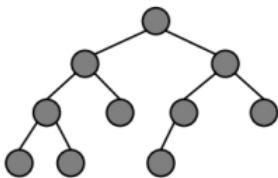
At the last level, all the nodes are as far left as possible.
far left as possible 왼쪽으로 정렬이 되어 있다.

Balanced Binary Tree

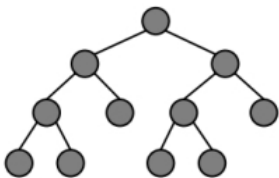
A binary tree where the heights of the left and right subtrees of every node differ by at most 1.

Perfect Binary Tree

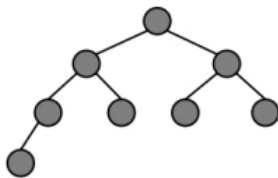
A binary tree where all internal nodes have 2 child nodes.
All leaf nodes are at the same level (depth).
딱딱 다 차 있으면서, 모든 leaf노드가 같은 level에 있음.



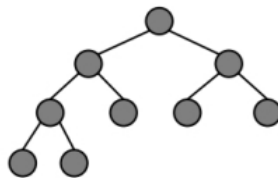
Neither full nor complete



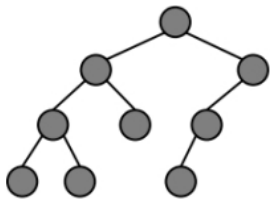
Full but not complete



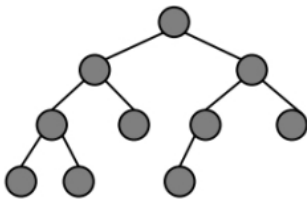
Complete but not full



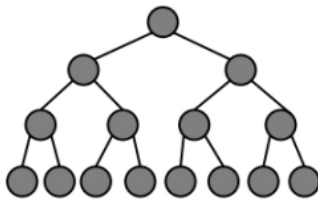
Full and complete



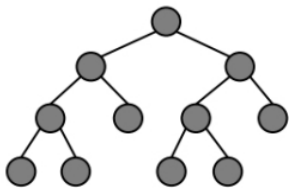
Neither perfect nor balanced



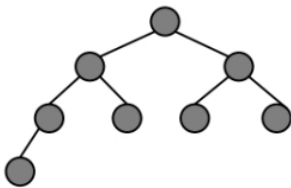
Balanced but not perfect



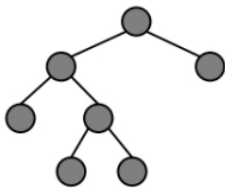
Perfect and balanced



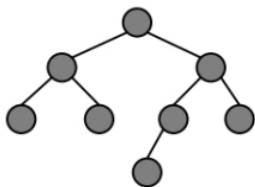
(a)



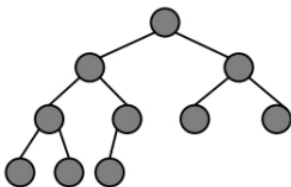
(b)



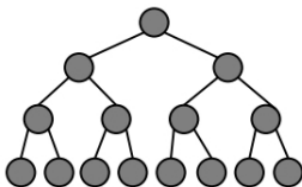
(c)



(d)



(e)



(f)

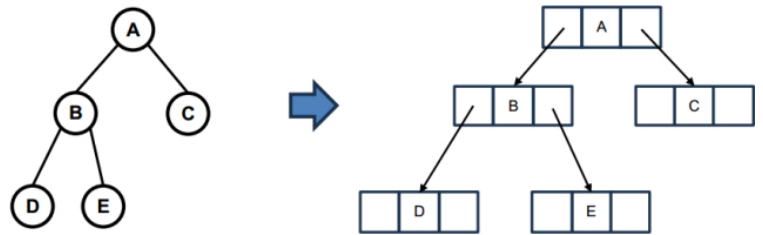
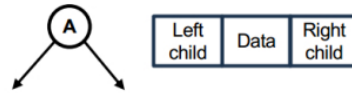
트리	Full	Complete	Balanced	Perfect
a	O	X	O	X
b	X	O	O	X
c	O	X	O	X
d	X	X	O	X
e	X	X	O	X
f	O	O	O	O

- The maximum number of nodes at level $i = 2^i$
- The maximum number of nodes in depth h binary tree
 - $1 + 2 + \dots + 2^h = 2^{h+1} - 1$
- The minimum number of nodes in n node binary tree
 - $\log(n + 1) - 1$, since $n = 2^{h+1} - 1$
- The number of edges in n node binary tree
 - $n - 1$

Binary Tree의 표현

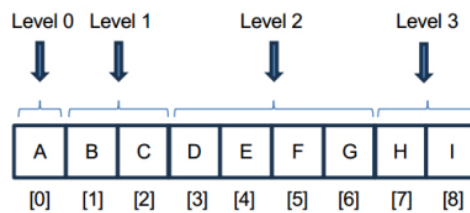
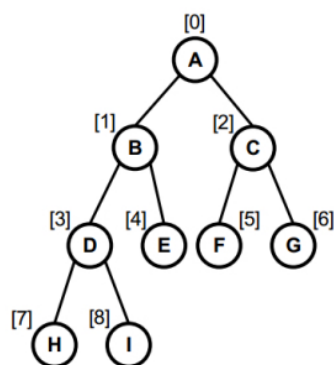
1. Linked list

```
typedef struct tNode {
    int data;
    struct tNode* left_node;
    struct tNode* right_node;
} tNode;
```



2. Array

A complete binary tree can be represented by an array
complete binary tree는 array로 나타낼 수 있다!



$$\text{parent}(i) = \lfloor i/2 \rfloor - 1$$

$$\text{leftChild}(i) = 2i + 1 \quad (2i + 1 < n)$$

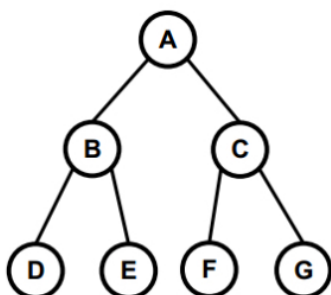
$$\text{rightChild}(i) = 2i + 2 \quad (2i + 2 < n)$$

Tree Traversal

DFS Search : In-order Traversal, Pre-order Traversal, Post-order Traversal

In-order-Traversal LVR

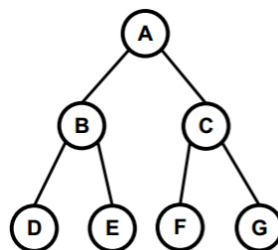
Left subtree -> Visit node -> Right subtree



D → B → E → A → F → C → G

Pre-order Traversal VLR

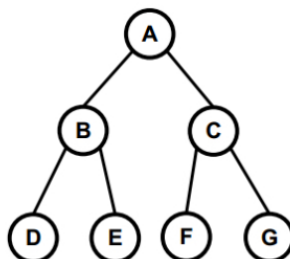
Visit node -> Left subtree -> Right subtree



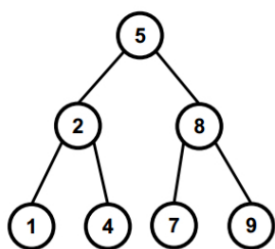
A → B → D → E → C → F → G

Post-order Traversals LRV

Left subtree -> Right subtree -> Visit node



D → E → B → F → G → C → A



In-order: 1 → 2 → 4 → 5 → 7 → 8 → 9

Pre-order: 5 → 2 → 1 → 4 → 8 → 7 → 9

Post-order: 1 → 4 → 2 → 7 → 9 → 8 → 5

Binary Search Tree 이진 탐색 트리

Binary Search Tree is a binary tree, in which for **every node**,

이진 탐색 트리는 이진 트리인데, 모든 노드에 대해

the **right subtree** contains only nodes with values larger than the node's value

오른쪽 서브트리는 해당 노드보다 큰 값만 가진다

and **left subtree** contains only nodes with values smaller than the node's value

왼쪽 서브트리는 해당 노드보다 작은 값만 가진다

평균 탐색 시간 : **O(logn)** - 탐색이 빠르다

Storing Sorted Data with Fast Access 정렬된 데이터 저장에 적합하다

In-order traversal for sorted output

정렬하기 위해서 in-order를 해야 함

Search(x, T)

```

BinaryTreeNode* findMin(BinaryTreeNode* root) {
    if (root == NULL)
        return NULL;
    else if (root->left != NULL)
        return findMin(root->left);
    return root;
}

BinaryTreeNode* findMax(BinaryTreeNode* root) {
    if (root == NULL)
        return NULL;
    else if (root->right != NULL)
        return findMax(root->right);
    return root; }
  
```

```

BinaryTreeNode* searchNode(BinaryTreeNode* root, int target) {
    if (root == NULL)
        return NULL;
    if (root->key > target)
        return searchNode(root->left, target);
    else if (root->key < target)
        return searchNode(root->right, target);
    else
        return root;
}

```

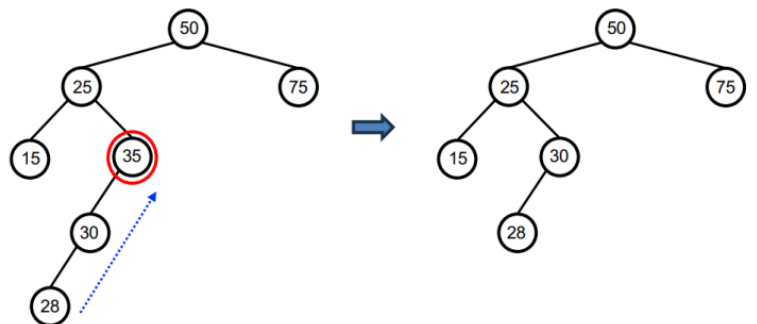
```

BinaryTreeNode* insertNode(BinaryTreeNode* node, int value) {
    if (node == NULL) {
        BinaryTreeNode* temp = (BinaryTreeNode*)malloc(sizeof(BinaryTreeNode));
        temp->key = value;
        temp->left = temp->right = NULL;
        return temp;
    }
    if (value < node->key) // 맥락상 node->key는 노드에 저장된 값
        node->left = insertNode(node->left, value);
    else if (value > node->key)
        node->right = insertNode(node->right, value);
    else
        // the key value is already in BST
        ; // do nothing
    return node;
}

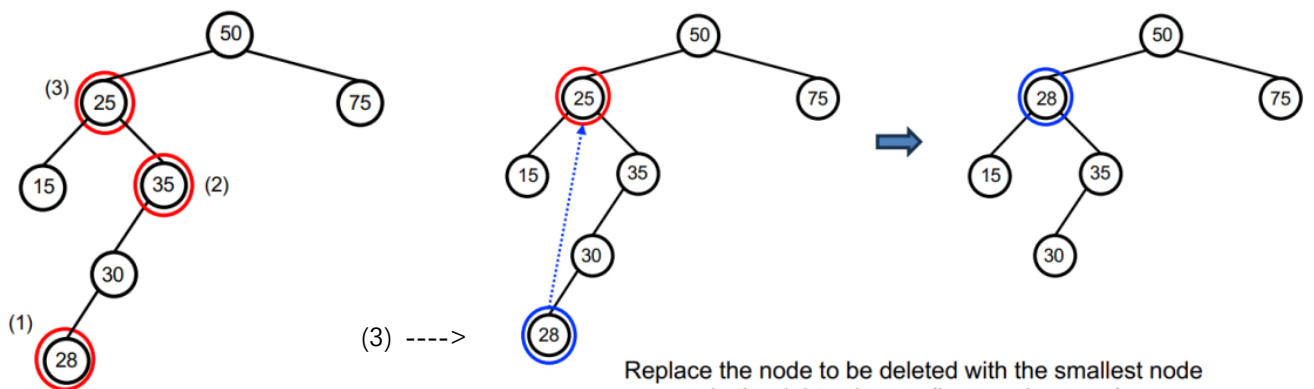
```

Delete

- (1) child가 없을 때 -> 그냥 지우기
- (2) child가 1개일 때 (2) ---->
- (3) child가 2개일 때



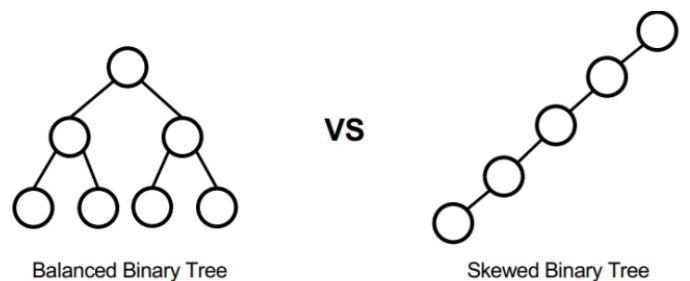
Delete the node and plug the child into the spot where the deleted node was



Replace the node to be deleted with the smallest node in the right subarray (i.e., predecessor)

BST 복잡도

- Best case : $O(\log n)$ (complete binary tree)
- Worst case : $O(n)$ (skewed binary tree)



Balanced Binary Tree

Skewed Binary Tree

시험 포인트

1. TF문제 20문항 있음

맞으면 +1 틀리면 -1

e.g. Asymtotic analysis is ~...

세세한(마이너한) 것이라도 틀리면 False로 체크해야 함.

2. 코드를 주고 몇명이서 코드에 대해서 이야기를 할 것임.

다음 중 맞는 이야기를 하는 사람을 모두 고르시오.

3. Stack Queue

Postfix 계산하라! **Postfix** 계산

Infix -> Postfix 변환

토큰이 어떻게 쌓이는지, top index가 어떻게 변하는지 적기

4. 코드 일부분을 보여주고

다음 중 맞는 이야기를 하는 사람을 모두 고르시오.

5. 코드 일부분을 보여주고

빈칸을 채우시오

6. Full / Complete / Balanced / Perfect 구분

이게 full인지, complete인지, balanced인지, perfect인지 판단

그리고 왜 그렇게 판단했는지 reason까지 써야 한다

이유 틀리면 0점

7. Tree Traversal

In-order, Pre-order, Post-order

하나라도 틀리면 0점

8. 트리 유형 분류 + 이유 쓰기

full / complete / balanced / perfect

이유 틀리면 0점